



Università
degli Studi di
Messina

Università degli Studi di Messina

Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Intelligenza Artificiale

PROGETTO 1:

Segmentazione Clienti e Churn Prediction

Massimo Mantineo

Matricola: 541924

Messina, A.A. 2025/2026

Indice

1	Descrizione del problema	3
1.1	Il Task Non Supervisionato (Clustering)	3
1.2	Il Task Supervisionato (Classificazione)	3
1.3	Le Metriche di Valutazione della Classificazione	3
2	Dataset e preparazione dei dati	4
2.1	Struttura e Principali Variabili	4
2.2	Operazioni di Preprocessing e Risposte Metodologiche	4
2.2.1	Gestione dei valori mancanti e Imputazione	4
2.2.2	Codifica delle variabili categoriche (One-Hot Encoding)	5
2.2.3	Standardizzazione (Standard Scaling)	5
2.3	Criticità: Sbilanciamento delle Classi	6
3	Metodologia	6
3.1	I Modelli Scelti	7
3.1.1	k-Means (Clustering)	7
3.1.2	Regressione Logistica (Baseline)	7
3.1.3	Decision Tree (Modello Principale 1)	7
3.1.4	Multi-Layer Perceptron (Modello Principale 2)	7
3.2	Riproducibilità, Dipendenze e Istruzioni d'Esecuzione	8
4	Esperimenti	8
4.1	Setup Sperimentale e Prevenzione dell'Overfitting	8
4.2	Caratterizzazione dei due Cluster ($k = 2$)	9
5	Risultati	9
5.1	Risultati del Clustering k-Means	9
5.2	Risultati della Classificazione (Train vs. Test)	10
5.3	Analisi dei Risultati e dell'Overfitting	10
6	Discussione critica	12
6.1	Punti di Forza della Soluzione	12
6.2	Limiti Sperimentali e Fonti di Errore	12
6.3	Confronto: Decision Tree vs. MLP	12
6.4	Miglioramenti Futuri	13
6.5	Uso di strumenti di Intelligenza Artificiale Generativa	13
7	Conclusioni	13

Sommario

In questo progetto ho studiato come unire il clustering (apprendimento non supervisionato) e la classificazione (apprendimento supervisionato) per analizzare il comportamento dei clienti di una compagnia telefonica (usando il dataset Telco Customer Churn). L'obiettivo principale è prevedere quali clienti rischiano di abbandonare l'operatore (churn prediction). Prima ho diviso i clienti in due gruppi (cluster) con l'algoritmo k-Means, trovando un profilo di clienti fedeli che spendono poco e uno di clienti a rischio che spendono molto. Poi ho addestrato tre modelli per la classificazione: la Regressione Logistica, un Albero di Decisione e una Rete Neurale (MLP). Ho confrontato le prestazioni dei modelli con e senza la feature del cluster. La Regressione Logistica e l'Albero hanno mostrato prestazioni stabili (80.6% accuratezza), mentre la rete neurale MLP ha avuto un forte overfitting. Tuttavia, aggiungendo il cluster come feature, la Recall (cioè la capacità di individuare chi abbandona davvero) dell'MLP sul test set è salita dal 42.8% al 58.3% (+15.5%). Nelle conclusioni discuto l'interpretabilità dei modelli, i loro limiti e le possibili migliorie come il bilanciamento dei dati.

1 Descrizione del problema

Per un'azienda telefonica, perdere un cliente (fenomeno chiamato *customer churn* o abbandono) è un grosso problema economico. Trovare nuovi clienti costa molto più che tenersi quelli già attivi. Per questo è utile costruire un sistema che possa prevedere in anticipo se un cliente sta per andarsene, in modo che l'azienda possa fargli un'offerta e convincerlo a restare.

Questo progetto affronta il problema usando due approcci del Machine Learning in una pipeline integrata: il clustering (non supervisionato) e la classificazione (supervisionata).

1.1 Il Task Non Supervisionato (Clustering)

Con il clustering vogliamo dividere i clienti in gruppi (o segmenti) in base a quanto si somigliano nei loro comportamenti (costi, tipo di contratto, mesi di permanenza). L'apprendimento è “non supervisionato” perché il computer fa questa divisione da solo, senza sapere in anticipo chi ha abbandonato o meno la compagnia.

La segmentazione dei clienti viene qui affrontata con l'algoritmo *k*-Means, provando a dividere i clienti in 2, 3 o 4 gruppi. Per scegliere il numero migliore di gruppi uso due indicatori:

- **Inertia (Metodo del Gomito)**: misura quanto i punti sono vicini al centro del proprio gruppo. Più il valore è basso, più i gruppi sono compatti internamente.
- **Silhouette Score**: misura quanto un cliente è simile al proprio gruppo rispetto a quanto sia dissimile dagli altri. Varia tra -1 e +1; valori più alti indicano gruppi ben separati e definiti.

1.2 Il Task Supervisionato (Classificazione)

Il secondo compito è una classificazione binaria. Vogliamo addestrare un modello che, guardando le caratteristiche di un cliente (come contratti, costi e il gruppo in cui è stato inserito), riesca a prevedere se abbandonerà la compagnia (classe 1, *Churn: Sì*) o rimarrà fedele (classe 0, *Churn: No*).

1.3 Le Metriche di Valutazione della Classificazione

Per capire se i modelli funzionano bene, ho usato tre metriche classiche:

- **Accuracy (Accuratezza)**: ci dice semplicemente quante predizioni sono corrette sul totale delle prove. È utile ma può ingannare se le classi sono molto sbilanciate:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

dove *TP*, *TN*, *FP* e *FN* sono rispettivamente i Veri Positivi, Veri Negativi, Falsi Positivi e Falsi Negativi ricavati dalla matrice di confusione.

- **Recall (Richiamo)**: ci dice quanti dei clienti che vogliono abbandonare davvero (veri positivi) vengono scoperti dal modello:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

Nel nostro caso, questa è la metrica più importante per il business. Se il modello non vede un cliente che sta per andarsene (falso negativo), l'azienda lo perde senza poter fare nulla per trattenerlo.

- **AUC (Area Under the ROC Curve):** misura la capacità del modello di distinguere tra le due classi a prescindere dalla soglia scelta. Varia tra 0.5 (scelte casuali, come lanciare una moneta) e 1.0 (modello perfetto). È una misura molto robusta quando le classi sono sbilanciate.

2 Dataset e preparazione dei dati

Ho usato il dataset *Telco Customer Churn* fornito da IBM (si trova facilmente su Kaggle). In totale contiene 7043 clienti, descritti da 21 caratteristiche (colonne).

2.1 Struttura e Principali Variabili

Le caratteristiche si dividono in tre categorie principali:

1. **Variabili numeriche continue:** mesi da cui il cliente è attivo (*tenure*), spesa mensile (*MonthlyCharges*) e spesa totale (*TotalCharges*).
2. **Variabili categoriche qualitative:** tipo di contratto (*Contract*: mensile, annuale, biennale), metodo di pagamento (*PaymentMethod*), tipo di internet (*InternetService*) e altri servizi accessori (es. backup online, streaming tv).
3. **Variabili categoriche binarie (sì/no):** genere, se ha un partner o familiari a carico, se usa la bolletta online e se è un cliente anziano (*SeniorCitizen*, codificata numericamente in $\{0, 1\}$).

La variabile da prevedere è **Churn**, che assume valore “Yes” se il cliente ha abbandonato la compagnia nell'ultimo mese, “No” in caso contrario.

2.2 Operazioni di Preprocessing e Risposte Metodologiche

Prima di addestrare i modelli ho dovuto sistemare e preparare i dati con alcuni passaggi fondamentali.

2.2.1 Gestione dei valori mancanti e Imputazione

Nella colonna *TotalCharges* c'erano 11 righe con spazi vuoti. Andando a controllare, ho notato che tutti questi clienti avevano *tenure* pari a 0 mesi (erano clienti appena registrati che non avevano ancora ricevuto nessuna bolletta). Per questo motivo, invece di cancellarli o usare valori calcolati, ho scelto di inserire semplicemente 0.0.

Che cos'è l'imputazione con la mediana e come si calcola?

Se avessimo voluto riempire i dati mancanti usando la mediana, avremmo ordinato tutti i valori dal più piccolo al più grande e preso quello centrale (o la media dei due centrali se il numero totale è pari).

Matematicamente, per calcolare il valore mediano: 1. Si ordinano gli N valori della variabile in ordine non decrescente: $x_1 \leq x_2 \leq \dots \leq x_N$. 2. Se N è dispari, la mediana è il valore centrale in posizione $\frac{N+1}{2}$:

$$\text{Mediana} = x_{\frac{N+1}{2}} \quad (3)$$

3. Se N è pari, la mediana è la media aritmetica dei due valori centrali in posizione $\frac{N}{2}$ e $\frac{N}{2} + 1$:

$$\text{Mediana} = \frac{x_{\frac{N}{2}} + x_{\frac{N}{2}+1}}{2} \quad (4)$$

Perché si usa la mediana invece della media?

La mediana è ottima perché è robusta contro i valori estremi (outlier), a differenza della media che risente pesantemente di un singolo valore molto alto o molto basso. Nel nostro caso, però, mettere 0.0 era la scelta logicamente corretta per il business, mentre usare la mediana generale (circa 1397.47\$) sarebbe stato un errore perché avrebbe assegnato una spesa alta a persone che non sono ancora clienti da un mese.

2.2.2 Codifica delle variabili categoriche (One-Hot Encoding)

I modelli matematici non capiscono le parole (come “Mensile” o “Annuale”). Perciò ho usato il One-Hot Encoding:

- **In cosa consiste?**

Consiste nel mappare una variabile qualitativa con C categorie distinte in C variabili binarie (aventi valore 1 se l'osservazione appartiene alla categoria e 0 altrimenti).

- **Perché escludiamo una categoria (`drop_first=True`)?**

Per evitare problemi matematici (multicollinearità). Se tenessimo tutte le C colonne, la loro somma darebbe sempre 1, creando una dipendenza lineare perfetta con il termine di intercetta (bias) dei modelli lineari, il che rende impossibile invertire le matrici nei calcoli. Escludendo la prima colonna, questa funge da riferimento. Ad esempio, per il tipo di contratto (mensile, annuale, biennale), bastano due colonne (annuale e biennale): se entrambe sono 0, significa che il contratto è mensile.

- **Come verifichiamo la correttezza nel codice?**

Ho usato la funzione `pd.get_dummies()` di pandas. Per verificare:

1. Ho controllato le dimensioni della tabella (il numero di colonne è passato da 20 a 30).
2. Ho guardato le prime righe per verificare che ci fossero solo 0 e 1 nelle nuove colonne.

2.2.3 Standardizzazione (Standard Scaling)

I modelli basati sulle distanze (k -Means) e sulle reti neurali (MLP) risentono fortemente delle differenze di scala tra le variabili.

- **Come funziona lo Standard Scaling?**

La standardizzazione trasforma le variabili numeriche continue in modo che abbiano

tutte media $\mu = 0$ e deviazione standard $\sigma = 1$ (la deviazione standard indica quanto in media i valori si allontanano dal valore medio). Per ciascun valore x_i , si calcola il valore scalato z_i :

$$z_i = \frac{x_i - \mu}{\sigma} \quad (5)$$

dove:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (\text{Media}) \quad (6)$$

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \quad (\text{Deviazione Standard}) \quad (7)$$

- **Perché lo splitting in Train/Test si fa PRIMA dello scaling?**

Questa è una regola fondamentale per evitare il **Data Leakage** (trapelamento di informazioni). Se calcolassimo la media μ e la deviazione standard σ su tutto il dataset prima di dividerlo, le informazioni del test set (che deve simulare dati futuri sconosciuti) influenzerebbero i valori del train set. Perciò ho diviso prima il dataset, calcolato i parametri di scaling solo sul Train, e infine li ho applicati a entrambi.

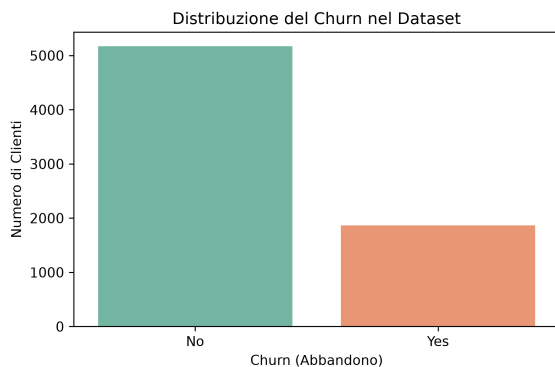


Figura 1: Distribuzione del Churn nel dataset.

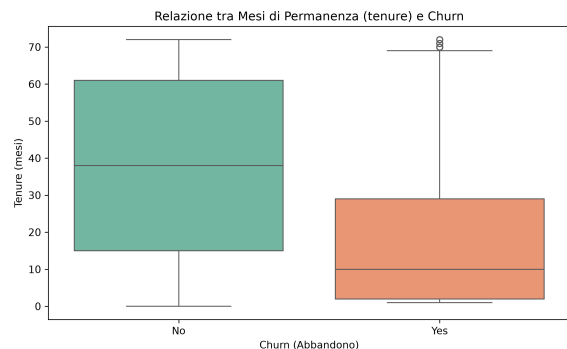


Figura 2: Relazione tra mesi di permanenza (tenure) e Churn.

2.3 Criticità: Sbilanciamento delle Classi

Il dataset è molto sbilanciato: il 73.4% dei clienti non ha abbandonato (classe 0), mentre solo il 26.6% ha abbandonato (classe 1). C'è un rapporto di circa 3 a 1. Se non si fa attenzione, un modello potrebbe limitarsi a dire sempre "No Churn" e ottenere un'accuratezza del 73.4% senza capire nulla del problema. Per questo ho eseguito uno split train-test stratificato, per mantenere la stessa proporzione di clienti a rischio sia nel set di addestramento che in quello di test.

3 Metodologia

La mia pipeline di lavoro è divisa in tre parti: 1. Pulizia dei dati, codifica delle categoriche e standardizzazione. 2. Applicazione di k -Means per generare la feature del cluster e aggiunta di questa colonna al dataset. 3. Addestramento dei tre classificatori (Regressione

Logistica, Albero di Decisione, MLP) sia con che senza la colonna del cluster, per valutarne l'effetto.

3.1 I Modelli Scelti

3.1.1 k-Means (Clustering)

L'algoritmo k -Means serve a dividere i dati in gruppi. All'inizio sceglie a caso dei punti centrali (centroidi) per ogni gruppo. Poi ripete due passi:

1. Assegna ogni cliente al gruppo il cui centroide è più vicino (usando la distanza euclidea).
2. Ricalcola il centro di ogni gruppo facendo la media delle posizioni dei punti assegnati ad esso.

Si ferma quando i centri non si muovono più. Ho provato soluzioni con 2, 3 e 4 cluster valutando Inertia e Silhouette Score per scegliere il numero ottimale di gruppi.

3.1.2 Regressione Logistica (Baseline)

Usata come baseline semplice. Calcola una somma pesata delle caratteristiche del cliente e la passa dentro la funzione sigmoide:

$$\sigma(t) = \frac{1}{1 + e^{-t}} \quad (8)$$

Questa funzione schiaccia il risultato in un valore tra 0 e 1, che rappresenta la probabilità che il cliente abbandoni.

3.1.3 Decision Tree (Modello Principale 1)

Questo modello ragiona a domande sì/no (regole). Sceglie le suddivisioni migliori per separare il più possibile i clienti a rischio da quelli fedeli, misurando la purezza dei gruppi tramite l'indice di Gini. Ho impostato la profondità massima dell'albero a 5 (`max_depth=5`) per evitare che cresca troppo e impari a memoria le risposte (overfitting).

3.1.4 Multi-Layer Perceptron (Modello Principale 2)

Il Multi-Layer Perceptron (MLP) è una rete neurale artificiale composta da vari strati di neuroni collegati tra loro.

L'addestramento funziona in due fasi ripetute iterativamente:

1. **Fase in avanti (Forward Propagation):** I dati del cliente entrano nella rete, passano per gli strati nascosti (dove i neuroni calcolano somme pesate e applicano una funzione di attivazione non lineare come la ReLU, $f(x) = \max(0, x)$) e arrivano all'uscita.
2. **Fase all'indietro (Backpropagation):** Si calcola l'errore tra la risposta della rete e quella reale (usando la perdita di cross-entropia). L'errore viene propagato a ritroso per correggere i pesi dei collegamenti usando un ottimizzatore (Adam) basato sulla discesa del gradiente.

Perché nei neuroni nascosti servono funzioni non lineari?

Senza di esse, la composizione di più strati lineari collasserebbe matematicamente in una singola trasformazione lineare. La rete neurale perderebbe così la capacità di apprendere relazioni complesse, diventando equivalente ad un semplice modello di Regressione Logistica.

Perché nell'ultimo neurone serve la Sigmoidale?

Perché vogliamo che l'output sia una probabilità compresa tra 0 e 1. Se non la usassimo, la rete darebbe in uscita un numero reale qualsiasi, rendendo impossibile interpretarlo come probabilità e calcolare l'errore con la cross-entropia standard per la classificazione binaria.

3.2 Riproducibilità, Dipendenze e Istruzioni d'Esecuzione

Per rendere il progetto riproducibile, tutto il codice è racchiuso nel notebook `segmentazione_crunch.ipynb`. Se il dataset non è presente localmente nella cartella `dati/`, il notebook lo scarica da solo all'avvio.

Le librerie usate con le rispettive versioni di riferimento sono:

- **Pandas** e **NumPy** per gestire le tabelle e i calcoli matematici.
- **Scikit-Learn** per il clustering (*k*-Means) e i modelli di classificazione.
- **Matplotlib** e **Seaborn** per disegnare i grafici.

Le dipendenze esatte sono raccolte nel file `requirements.txt` e le istruzioni di installazione sono spiegate nel file `README.md` nella radice del progetto.

4 Esperimenti

4.1 Setup Sperimentale e Prevenzione dell'Overfitting

Per verificare che i modelli funzionino bene su dati nuovi, dobbiamo controllare due problemi:

- **Overfitting (imparare a memoria)**: succede quando il modello si adatta troppo bene ai dati di addestramento (compreso il rumore e i dettagli inutili), ma poi fa molti errori sui dati nuovi del Test Set.
- **Underfitting (troppo semplice)**: succede quando il modello è troppo semplice e non riesce a capire le regole del problema, ottenendo risultati scarsi sia sul Train e sul Test.

Per verificare questi comportamenti, ho addestrato i modelli sull'80% dei dati (Train) e poi li ho valutati sul restante 20% (Test), confrontando le metriche ottenute nei due casi per essere sicuro che le prestazioni fossero stabili.

4.2 Caratterizzazione dei due Cluster ($k = 2$)

Dopo aver eseguito l'algoritmo k-Means con $k = 2$, ho analizzato le caratteristiche dei clienti nei due gruppi (usando i valori originali prima della standardizzazione):

1. Cluster 0 (1526 clienti - “Fedeli e con spesa bassa”):

- Tenure media: 30.55 mesi.
- Spesa mensile media (*MonthlyCharges*): 21.08\$.
- Costo totale medio (*TotalCharges*): 662.60\$.
- Tasso di Churn reale: **7.40%** (estremamente basso).
- Contratti: Prevalenza di contratti a lungo termine (Biennali: 41.8%, Annuali: 23.9%, Mensili: 34.3%).
- *Profilo*: Clientela stabile che utilizza servizi di base, caratterizzata da elevata fedeltà e contratti pluriennali.

2. Cluster 1 (5517 clienti - “A rischio e con spesa alta”):

- Tenure media: 32.88 mesi.
- Spesa mensile media (*MonthlyCharges*): 76.84\$.
- Costo totale medio (*TotalCharges*): 2727.03\$.
- Tasso di Churn reale: **31.83%** (molto alto).
- Contratti: Forte prevalenza di contratti flessibili mensili (Mese-a-mese: 60.7%).
- *Profilo*: Clientela ad alto consumo tecnologico (banda larga, streaming), ma molto volatile e legata a contratti mensili senza vincoli temporali.

5 Risultati

5.1 Risultati del Clustering k-Means

Ho testato k-Means con 2, 3 e 4 cluster. I risultati di Inertia e Silhouette Score sono riportati nella Tabella 1.

Tabella 1: Metriche di valutazione del clustering k-Means.

Numero di Cluster (k)	Inertia	Silhouette Score
2	41817.82	0.2946
3	30802.81	0.2861
4	28228.08	0.2279

La silhouette score migliore si ottiene con $k = 2$ (0.2946). Con $k = 3$ scende leggermente a 0.2861, e con $k = 4$ cala ancora a 0.2279. Per questo ho scelto $k = 2$ come suddivisione ottimale dei clienti. La Figura 3 mostra l'andamento delle due metriche.

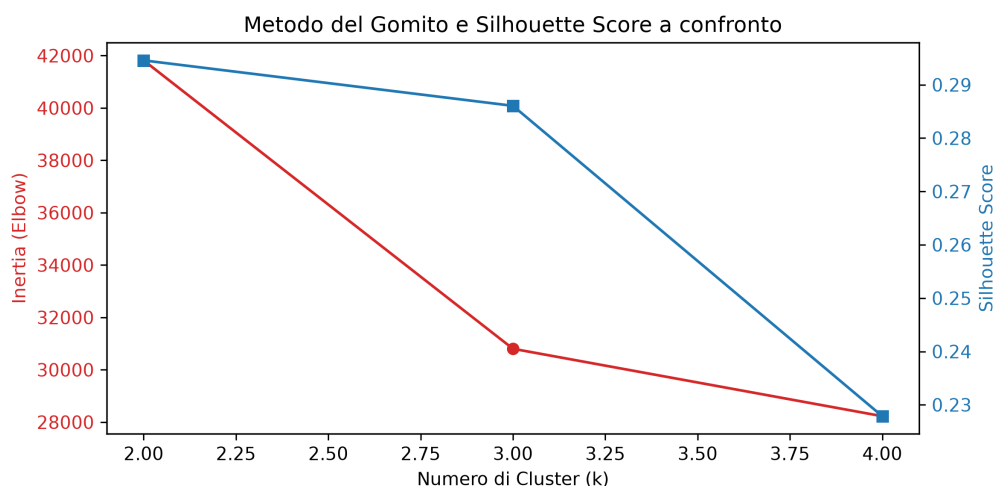


Figura 3: Valutazione del clustering k-Means tramite metodo del gomito (Inertia) e Silhouette Score.

5.2 Risultati della Classificazione (Train vs. Test)

Ho confrontato le prestazioni nei due scenari:

- **Scenario A (Senza Cluster):** usando solo le caratteristiche originali.
- **Scenario B (Con Cluster):** aggiungendo il cluster di appartenenza come caratteristica in più.

La Tabella 2 riassume le performance di tutti i modelli su Train e Test.

Tabella 2: Confronto delle metriche di classificazione tra Train e Test.

Modello	Feature Scenario	Accuracy		Recall		AUC	
		Train	Test	Train	Test	Train	Test
Logistic Regression	Senza Cluster	0.8053	0.8062	0.5485	0.5588	0.8492	0.8422
Logistic Regression	Con Cluster	0.8051	0.8055	0.5485	0.5588	0.8492	0.8423
Decision Tree	Senza Cluster	0.8023	0.7942	0.5612	0.5401	0.8476	0.8267
Decision Tree	Con Cluster	0.8023	0.7942	0.5612	0.5401	0.8476	0.8267
Multi-Layer Perceptron (MLP)	Senza Cluster	0.9143	0.7495	0.7813	0.4278	0.9656	0.7844
Multi-Layer Perceptron (MLP)	Con Cluster	0.9104	0.7360	0.9097	0.5829	0.9722	0.7846

5.3 Analisi dei Risultati e dell'Overfitting

Dall'ispezione della Tabella 2 emergono tre osservazioni importanti:

1. **Regressione Logistica e Albero di Decisione sono stabili:** Le prestazioni su Train e Test sono quasi identiche (es. accuratezza intorno all'80% per entrambi). Non c'è alcun segno di overfitting. L'albero di decisione funziona bene perché limitando la profondità massima a 5 gli abbiamo impedito di crescere troppo e memorizzare i dati di addestramento. In questi due modelli l'aggiunta del cluster non cambia i risultati, perché la divisione fatta dal clustering è lineare ed è già contenuta nelle altre variabili originali.

2. **La Rete Neurale (MLP) soffre di forte overfitting:** Sul Train set senza cluster l'MLP raggiunge risultati altissimi (Accuratezza 91.43%, AUC 0.9656), ma sul Test set crolla vistosamente (Accuratezza 74.95%, Recall 42.78%, AUC 0.7844). Essendo un modello molto complesso con molti parametri, tende a memorizzare i dettagli e il rumore del Train set invece di capire le regole generali.
3. **L'impatto del Cluster sull'MLP:** Pur non risolvendo l'overfitting, l'aggiunta del cluster come caratteristica nello Scenario B fa fare un grande balzo in avanti alla **Recall sul Test set, che sale dal 42.78% al 58.29% (+15.51% assoluto)**. Questo significa che il cluster aiuta la rete neurale a individuare molti più clienti a rischio di abbandono reale, anche se a costo di qualche falso positivo in più (l'accuratezza sul test scende leggermente al 73.60%).

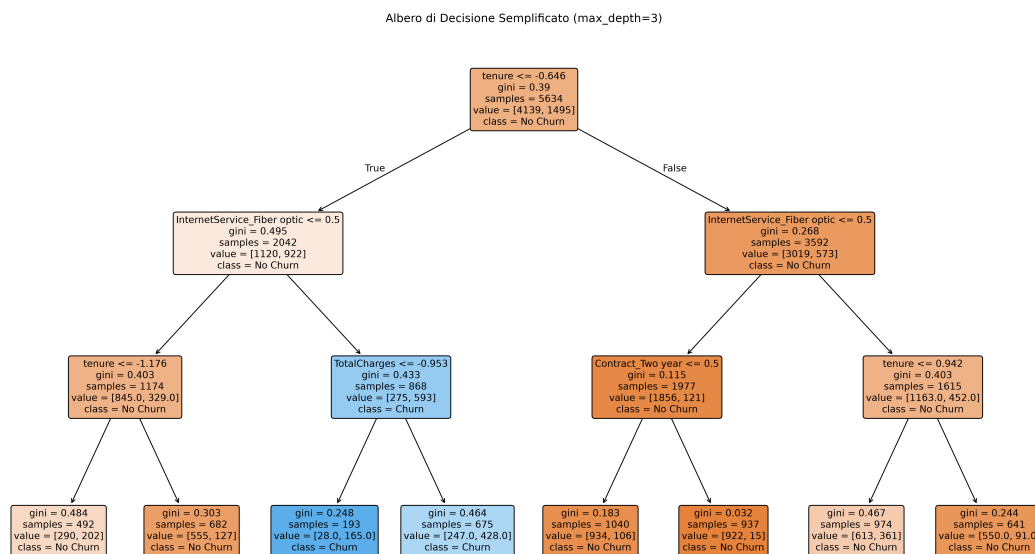


Figura 4: Albero di Decisione semplificato (max_depth=3).

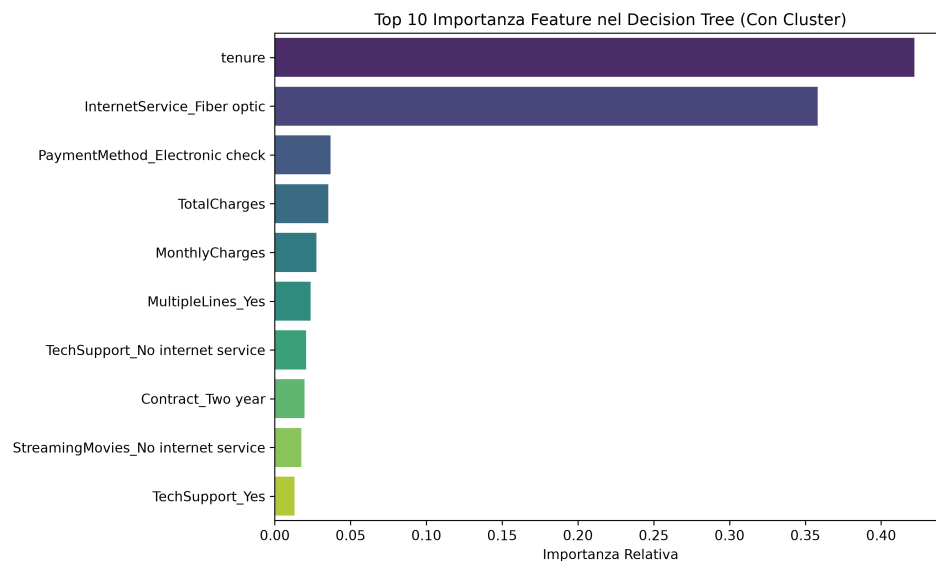


Figura 5: Top 10 importanza delle feature per il Decision Tree nello Scenario B (con feature del cluster).

6 Discussione critica

6.1 Punti di Forza della Soluzione

Unire il clustering non supervisionato e la classificazione ha permesso di descrivere chiaramente i profili commerciali dei clienti. Inoltre, l'aggiunta del cluster come caratteristica ha dato un aiuto concreto alla rete neurale MLP, aumentandone la Recall sul test set di oltre il 15%, un risultato molto utile per l'azienda.

6.2 Limiti Sperimentali e Fonti di Errore

I due cluster trovati hanno confini un po' sfumati (Silhouette Score di 0.2946), segno che i clienti non sono divisi in categorie nette ma hanno comportamenti intermedi. Inoltre, il forte overfitting dell'MLP dimostra che la rete neurale così com'è non è pronta per essere usata, e servirebbero tecniche per frenare questa tendenza (come l'aumento della regolarizzazione L2, il dropout o l'Early Stopping).

6.3 Confronto: Decision Tree vs. MLP

L'Albero di Decisione è un modello trasparente (*white-box*): guardando la Figura 4 si capiscono al volo le regole usate per classificare un cliente (la regola più importante è se il contratto è mensile e quanti mesi di permanenza ha il cliente). La Rete Neurale (MLP), invece, è una scatola nera (*black-box*): anche se offre una Recall migliore, è impossibile spiegare a parole come i pesi dei collegamenti arrivino a quella decisione, rendendola difficile da usare se l'azienda vuole capire i motivi precisi dietro le scelte dei clienti.

6.4 Miglioramenti Futuri

Per migliorare i modelli proporrei di:

- Usare tecniche come SMOTE (generazione di dati sintetici della classe minoritaria) o pesare di più la classe minore durante l'addestramento per bilanciare i dati.
- Provare modelli ensemble come Random Forest o XGBoost, che spesso gestiscono meglio l'overfitting rispetto alle reti neurali su dati tabellari.

6.5 Uso di strumenti di Intelligenza Artificiale Generativa

Per lo sviluppo di questo progetto ho utilizzato strumenti di intelligenza artificiale generativa (Large Language Models) solo per farmi aiutare a formattare il codice in $\text{\LaTeX}$ e a impostare lo scheletro della relazione.

7 Conclusioni

Il progetto ha dimostrato che unire apprendimento non supervisionato e supervisionato è una scelta vincente. Il clustering ha identificato due gruppi commerciali chiari (clienti fedeli che spendono poco contro clienti volatili che spendono molto). Aggiungere questa suddivisione come caratteristica ha aiutato a ridurre l'impatto dello sbilanciamento del dataset sulla Rete Neurale (MLP), portando a un incremento del +15.5% nel richiamo dei clienti a rischio di abbandono sul Test Set, anche se rimangono aperti i problemi di overfitting della rete che richiederanno modelli più regolarizzati in futuro.